

Jupyter Notebook and Data Collection

Dr. Amr M. Nagy

Faculty of computers and Artificial Intelligence

Benha University

Outline

- Python and Jupyter Notebook
- The data collection process
- Common data formats and handling
- Regular expressions and parsing

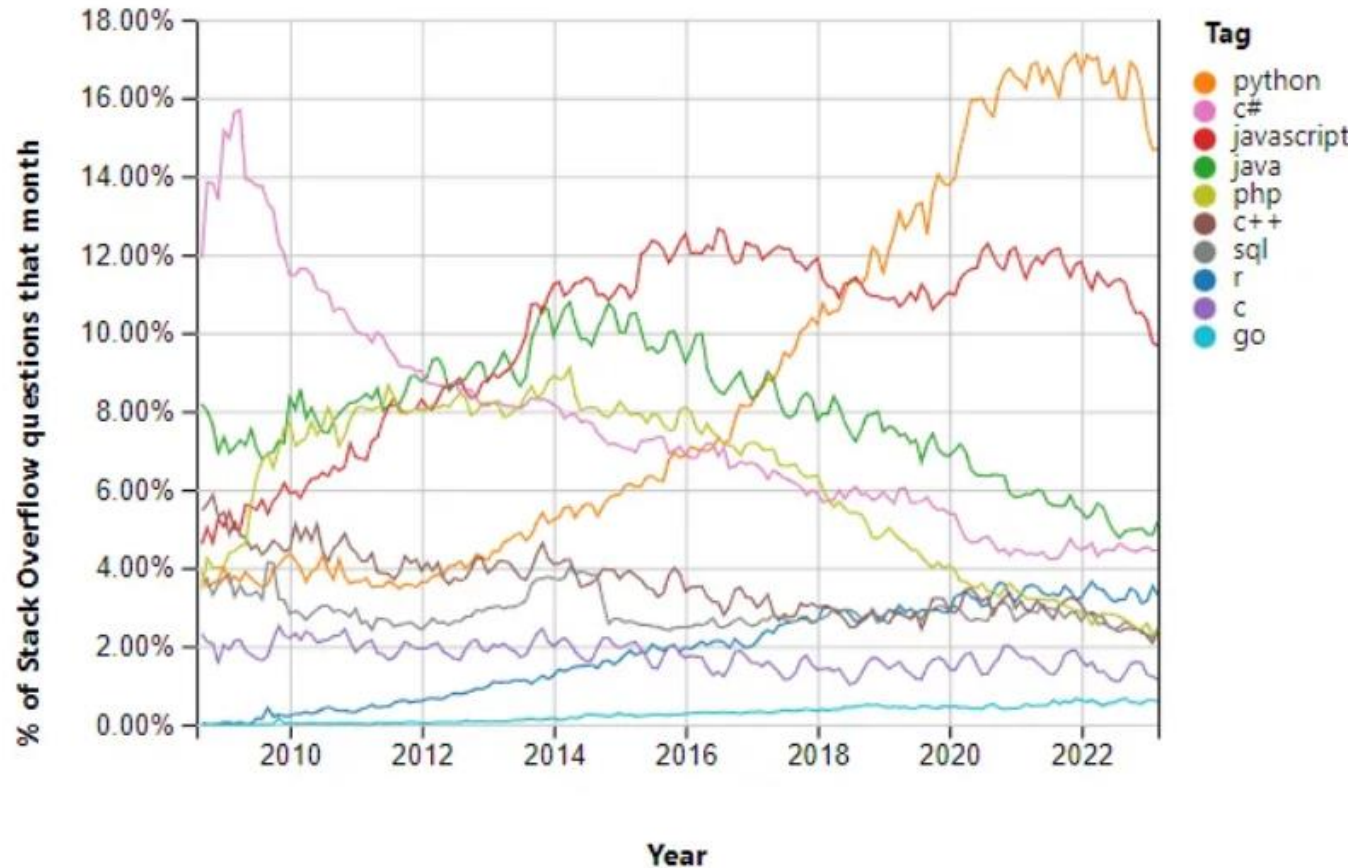
Python

- “The language of data science”
 - **Especially true** if the data science tasks involve lots of data processing and/or machine learning
 - **Less true** if the tasks are more “purely statistical” (then R is more standard)
- Python growth/popularity happening at the same time as the deep learning boom



Python growth

➤ Growth of major programming languages



StackOverflow Trends — April 2023 update

Source: <https://blog.devgenius.io/the-popularity-of-data-science-languages-python-and-r-april-2023-update-e9350ad15ffc>

Anaconda

- Anaconda, a common distribution of Python, which includes several common libraries and tools including the Jupyter notebook and a package manager, available at:

<https://www.anaconda.com/download/>

- You can verify you are using the Anaconda distribution by running Python and making sure you see something like the following:

```
$ python
Python 3.9.7 (default, Sep 16 2021, 08:50:36)
[Clang 10.0.0 ] :: Anaconda, Inc. on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Installing additional packages

- There are two typical ways to install these, via the conda package manager (part of Anaconda), and via pip:
 - `conda install beautifulsoup4` – install BeautifulSoup4
 - `conda search beautiful` – search conda packages for any that includes the string “beautiful”
- `pip install beautifulsoup4` – install BeautifulSoup4
- `pip search beautiful` – search pip packages for any that include the string “beautiful”
- Rule of thumb: use conda when you can (plays nicer with Anaconda installation), but some packages can only be installed via pip

Jupyter notebook

- Jupyter notebooks are a browser-based environment for writing code, interspersing code and Markdown, and displaying figures, all contained in “cells”
- More info about Jupyter here: <http://www.jupyter.org>
- Launch jupyter via the command:
 - `jupyter notebook`
 - Then it will print instructions to navigate to a specific localhost URL (a browser tab may even open automatically)
- You may also use Colab notebook to complete assignments fully online

Google Colab

- Google Colab (Colaboratory) is a Jupyter notebook environment that runs in the browser using Google Cloud.
- Easy to open *.ipynb files from Google Drive folders



The first step of data science

- **The first step in data science ...**
 - ... is to get some data
- **You will typically get data in one of four ways:**
 - **Directly download a data file (or files) manually**
 - **Query data from a database**
 - **Query an API (usually web-based, these days)**
 - **Scrap data from a webpage**

Overview of relational data

- The basic relation (i.e. the table)
- The term technical term “relation” can be interchanged with the standard notion we have of “tabular data,” say an instance of a “Person” relation

ID	Last Name	First Name	Role
1	Virtue	Pat	Instructor
2	Koppol	Pallavi	TA
3	Cordwell	Katherine	TA
4	Vajiac	Cat	TA
5	Veloso	Manuela	Student
6	Resnik	Judy	Student

Rows are called tuples (or records), represent a single instance of this relation, and must be unique

ID	Last Name	First Name	Role
1	Virtue	Pat	Instructor
2	Koppol	Pallavi	TA
3	Cordwell	Katherine	TA
4	Vajiac	Cat	TA
5	Veloso	Manuela	Student
6	Resnik	Judy	Student

Columns are called attributes, specify some element contained by each of the tuples

Multiple tables and relations

➤ Primary keys

Person

ID	Last Name	First Name	Role ID
1	Virtue	Pat	1
2	Koppol	Pallavi	2
3	Cordwell	Katherine	2
4	Vajiac	Cat	2
5	Veloso	Manuela	3
6	Resnik	Judy	3

Role

ID	Name
1	Instructor
2	TA
3	Student

- Primary key: unique ID for every tuple in a relation (i.e. every row in the table), each relation must have exactly one primary key

Multiple tables and relations

➤ Foreign keys

Person				Role	
ID	Last Name	First Name	Role ID	ID	Name
1	Virtue	Pat	1	1	Instructor
2	Koppol	Pallavi	2	2	TA
3	Cordwell	Katherine	2	3	Student
4	Vajiac	Cat	2		
5	Veloso	Manuela	3		
6	Resnik	Judy	3		

- A foreign key is an attribute that points to the primary key of another relation
- If you delete a primary key, need to delete all foreign keys pointing to it

Indexes (not indices)

- Indexes are created as ways to “quickly” access elements of a table
- For example, consider finding people with last name “Gates”: no option but just scan through the whole dataset: $O n$ operation

ID	Last Name	First Name	Role ID
1	Virtue	Pat	1
2	Koppol	Pallavi	2
3	Cordwell	Katherine	2
4	Vajiac	Cat	2
5	Veloso	Manuela	3
6	Resnik	Judy	3

Indexes

- Think of an index as a separate sorted table containing the indexed column and the tuple location: searching for value takes $O(\log n)$ time
- In practice, use data structure like a B-tree or several others

Person					Last Name Index	
Location	ID	Last Name	First Name	Role ID	Last Name	Location
0	1	Virtue	Pat	1	Cordwell	200
100	2	Koppol	Pallavi	2	Koppol	100
200	3	Cordwell	Katherine	2	Resnik	500
300	4	Vajiac	Cat	2	Vajiac	300
400	5	Veloso	Manuela	3	Veloso	400
500	6	Resnik	Judy	3	Virtue	0

- The primary key always has an index associated with it (so you can think of primary keys themselves as always being a fast way to access data)
- Indexes don't have to be on a single column, can have an index over multiple columns (with some ordering)

Entity relationships

- Several types of inter-table relationships
 - One-to-one
 - (One-to-zero/one)
 - One-to-many (and many-to-one)
 - Many-to-many
- These relate one (or more) rows in a table with one (or more) rows in another table, via a foreign key
- Note that these relationships are really between the “entities” that the tables represent, but we won’t formalize this beyond the basic intuition

One-to-many relationship

- We have already seen a one-to-many relationship: one role can be shared by many people, denoted as follows



Person

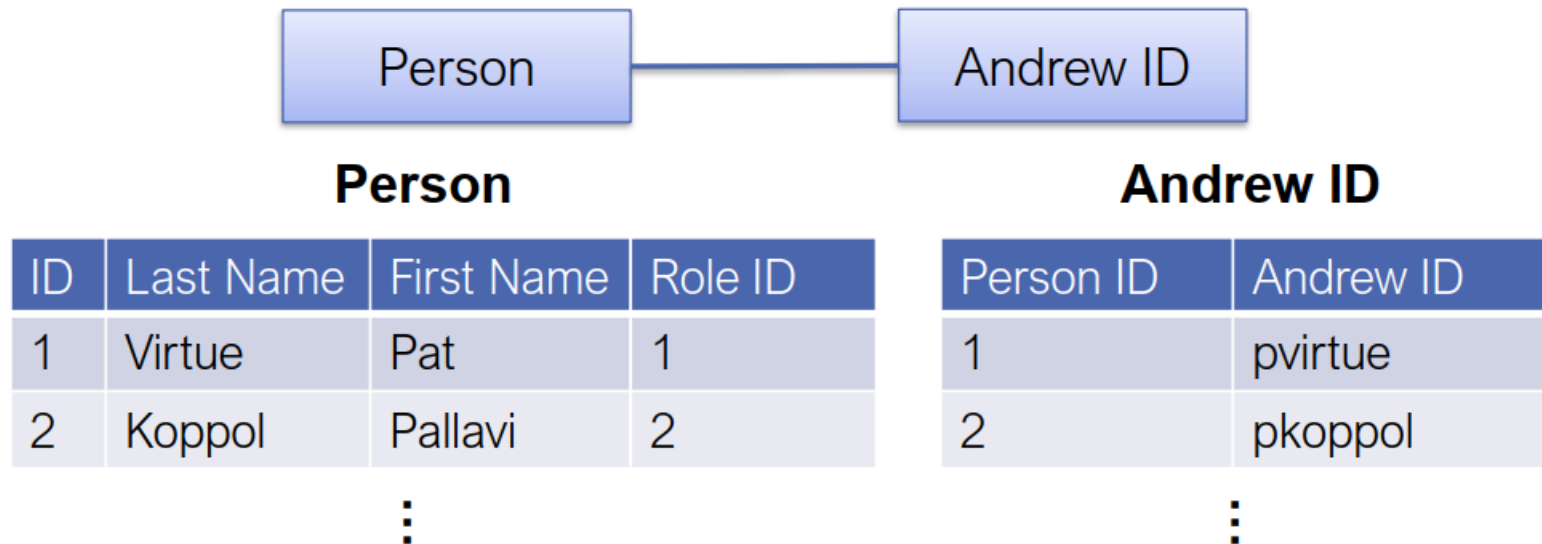
ID	Last Name	First Name	Role ID
1	Virtue	Pat	1
2	Koppol	Pallavi	2
3	Cordwell	Katherine	2
4	Vajiac	Cat	2
5	Veloso	Manuela	3
6	Resnik	Judy	3

Role

ID	Name
1	Instructor
2	TA
3	Student

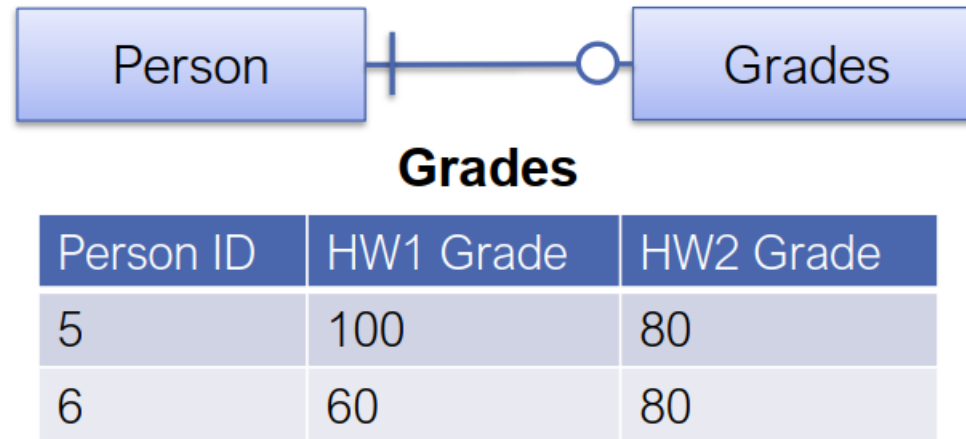
One-to-one relationships

- In a true one-to-one relationship spanning multiple tables, each row in a table has exactly one row in another table
- Not very common to break these across multiple tables, as you may as well just add another attribute to an existing table, but it is possible



One-to-zero/one relationships

- More common in databases is to find “one-to-zero/one” relationships broken across multiple tables
- Consider adding a “Grades” table to our database: each person can have at most one tuple in the grades table



- Bars and circles denote “mandatory” versus “optional” relationships (we won’t worry about these, just know that there is notation for them)

Many-to-many relationships

- Creating a grades table as done before is a bit cumbersome, because we need to keep adding columns to the table, null entries if someone doesn't do the homework
- Alternatively, consider adding two tables, a “homework” table that represents information about each homework, and an associative table that links homeworks to people

Homework

ID	Name	388 Points	688 Points
1	HW 1	65	35
2	HW 2	75	25

Person Homework

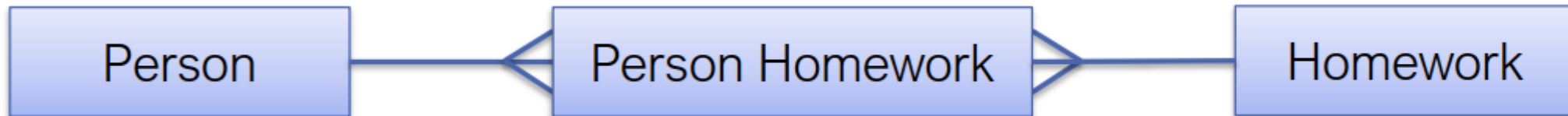
Person ID	HW ID	Score
5	1	100
5	2	80
6	1	60
6	2	80

Many-to-many relationships

- Setups like this encode many-to-many relationships: each person can have multiple homeworks, and each homework can be done by multiple people



- We could also write this in terms of relationships specified by the associative table, but this is not really correct, as it is mixing up the underlying relationships with how they are stored in a database



Pandas

- Pandas is a “Data Frame” library in Python, meant for manipulating in-memory data with row and column labels (as opposed to, e.g., matrices, that have no row or column labels)
- Pandas is not a relational database system, but it contains functions that mirror some functionality of relational databases
- We’re going to cover Pandas in more detail in other portions of the class, but just discuss basic functionality for now

Pandas examples

- Create a DataFrame with our Person example:

```
import pandas as pd

df = pd.DataFrame([(1, 'Virtue', 'Pat'),
                   (2, 'Koppol', 'Pallavi'),
                   (3, 'Cordwell', 'Katherine'),
                   (4, 'Vajiac', 'Cat'),
                   (5, 'Velooso', 'Manuela'),
                   (6, 'Resnik', 'Judy')],
                  columns=["id", "last_name", "first_name"])
```

	id	last_name	first_name
0	1	Virtue	Pat
1	2	Koppol	Pallavi
2	3	Cordwell	Katherine
3	4	Vajiac	Cat
4	5	Velooso	Manuela
5	6	Resnik	Judy

Some important notes

- As mentioned, Pandas is not a relational data system, in particular it has no notion of primary keys (but it does have indexes)
- Operations in Pandas are typically not in place (that is, they return a new modified DataFrame, rather than modifying an existing one)
 - Use the “inplace” flag to make them done in place
- If you select a single row or column in a Pandas DataFrame, this will return a “Series” object, which is like a one-dimensional DataFrame (it has only an index and corresponding values, not multiple columns)

Some common Pandas commands

```
# read CSV file into DataFrame
df = pd.read_csv(filename)

# get first five rows of DataFrame
df.head()

# index into a dataframe
# df.loc[rows, columns] and df.iloc[row numbers, column numbers]
df.loc[:, "Last Name"]           # Series of all last names
df.loc[:, ["Last Name"]]         # DataFrame with one column
df.loc[[1,2], :]                 # DataFrame with only ids 1,2
df.loc[1, "Last Name"] = "Vice"   # Set an entry in a DataFrame
df.loc[7, :] = ("Hebert", "Martial") # Add a new entry with index=7
df.iloc[0,0]                     # Index rows and columns by zero-index
```

➤ We're going to cover more next lecture in conjunction with visualization

SQLite

- An actual relational database management system (RDBMS)
- Unlike most systems, it is a serverless model, applications directly connect to a file
- Allows for simultaneous connections from many applications to the same database file (but not quite as much concurrency as client-server systems)
- All operations in SQLite will use SQL (Structured Query Language) command issued to the database object
- You can enforce foreign keys in SQLite, but we won't bother

Joins

- Join operations merge multiple tables into a single relation (can be then saved as a new table or just directly used)
- Four typical types of joins:
 - Inner
 - Left
 - Right
 - Outer
- You join two tables on columns from each table, where these columns specify which rows are kept

Example: joining Person and Grades

- Consider joining two tables, Person and Grades, on ID / Person ID

Person

ID	Last Name	First Name	Role ID
1	Virtue	Pat	1
2	Koppol	Pallavi	2
3	Cordwell	Katherine	2
4	Vajiac	Cat	2
5	Veloso	Manuela	3
6	Resnik	Judy	3

Grades

Person ID	HW1 Grade	HW2 Grade
5	100	80
6	60	80
100	100	100

Inner join (usually what you want)

- Join two tables where we only return the rows where the two joined columns contain the same value

ID	Last Name	First Name	Role ID	HW1 Grade	HW2 Grade
5	Veloso	Manuela	3	100	80
6	Resnik	Judy	3	60	80

- Only these two rows have an entry in “Person” and an entry in “Grades”

Inner join in Pandas/SQLite

- In Pandas, you can also join on index using `right_index/left_index` parameters
- There is also the `join` call in Pandas, which is a bit more limited (always assumes right is joined on index, left not on index)

```
# Pandas way
df_person = pd.read_sql_query("SELECT * FROM person", conn)
df_grades = pd.read_sql_query("SELECT * FROM grades", conn)
df_person.merge(df_grades, how="inner",
                left_on="id", right_on="person_id")

# SQLite way
cursor.execute("SELECT * FROM person, grades WHERE
               person.id == grades.person_id")
```

Left joins

- Keep all rows of the left table, add entries from right table that match the corresponding columns
- Example: left join Person and Grades on ID, Person ID

ID	Last Name	First Name	Role ID	HW1 Grade	HW2 Grade
1	Virtue	Pat	1	NULL	NULL
2	Koppol	Pallavi	2	NULL	NULL
3	Cordwell	Katherine	2	NULL	NULL
4	Vajiac	Cat	2	NULL	NULL
5	Veloso	Manuela	3	100	80
6	Resnik	Judy	3	60	80

Left join in Pandas and SQLite

```
# Pandas way
df_person.merge(df_grades, how="left",
                left_on="id", right_on="person_id")

# SQLite way
cursor.execute("SELECT * FROM person LEFT JOIN grades ON
               person.id == grades.person_id")
```

	id	last_name	first_name	person_id	hw1_grade	hw2_grade
0	1	Virtue	Pat	NaN	NaN	NaN
1	2	Koppol	Pallavi	NaN	NaN	NaN
2	3	Cordwell	Katherine	NaN	NaN	NaN
3	4	Vajiac	Cat	NaN	NaN	NaN
4	5	Veloso	Manuela	5.0	100.0	80.0
5	6	Resnik	Judy	6.0	60.0	80.0

Right join

- Like a left join but with the roles of the tables reversed

ID	Last Name	First Name	Role ID	HW1 Grade	HW2 Grade
5	Veloso	Manuela	3	100	80
6	Resnik	Judy	3	60	80
100	NULL	NULL	NULL	100	100

```
# Pandas way
df_person.merge(df_grades, how="right",
                left_on="id", right_on="person_id")

# Not supported in SQLite
```


Outer join

- Return all rows from both left and right join

ID	Last Name	First Name	Role ID	HW1 Grade	HW2 Grade
1	Virtue	Pat	1	NULL	NULL
2	Koppol	Pallavi	2	NULL	NULL
3	Cordwell	Katherine	2	NULL	NULL
4	Vajiac	Cat	2	NULL	NULL
5	Veloso	Manuela	3	100	80
6	Resnik	Judy	3	60	80
100	NULL	NULL	NULL	100	100

```
# Pandas way
df_person.merge(df_grades, how="outer",
                left_on="id", right_on="person_id")
```

Issuing HTTP queries

- The vast majority of automated data queries you will run will use HTTP requests (it's become the dominant protocol for much more than just querying web pages)
- (requests library, <http://docs.python-requests.org/>):

```
import requests
response = requests.get("http://www.datasciencecourse.org")

# some relevant fields
response.status_code
response.content # or response.text
response.headers
response.headers['Content-Type']
```

HTTP Request Basics

- You've seen URLs like these:
 - `https://www.google.com/url?sa=t&rct=j&q=&esrc=s&source=web&cd=9&cad=rja&uact=8...`
- The weird statements after the url are **parameters**, you would provide them using the requests library like this
- ... but sometimes the best way to solve a problem is just by visualizing the data, for instance

```
params = {"sa": "t", "rct": "j", "q": "", "esrc": "s",  
          "source": "web", "cd": "9", "cad": "rja", "uact": "8"}  
response = requests.get("http://www.google.com/url", params=params)
```

- HTTP GET is the most common method, but there are also PUT, POST, DELETE methods that change some state on the server

```
response = requests.put(...)  
response = requests.post(...)  
response = requests.delete(...)
```

RESTful APIs

- If you move beyond just querying web pages to web APIs, you'll most likely encounter REST APIs (Representational State Transfer)
- REST is more a design architecture, but a few key points:
 - Uses standard HTTP interface and methods (GET, PUT, POST, DELETE)
 - Stateless – the server doesn't remember what you were doing
- Rule of thumb: if you're sending your account key along with each API call, you're probably using a REST API

Querying a RESTful API

- You query a REST API similar to standard HTTP requests, but will almost always need to include parameters

```
token = "" # not going to tell you mine
headers = {'Authorization': 'token '+token}
response = requests.get("https://api.github.com/user", headers=headers)
print(response.content)
#{ "login": "pvirtue", "id": 5945661, "avatar_url": "https://avatars.githubu...
```

- Get your own access token at <https://github.com/settings/tokens/new>
- GitHub API uses GET/PUT/DELETE to let you query or update elements in your GitHub account automatically
- Example of REST: server doesn't remember your last queries, for instance you always need to include your access token if using it this way

Common data formats and handling

- Data formats
- The three most common formats (judging by my completely subjective experience):
 - CSV (comma separate value) files
 - JSON (Javascript object notation) files and strings
 - HTML/XML (hypertext markup language / extensible markup language) files and strings

CSV Files

- Refers to any delimited text file (not always separated by commas)

```
"Semester","Course","Section","Lecture","Mini","Last Name","Preferred/First  
Name","MI","Andrew ID","Email","College","Department","Class","Units","Grade  
Option","QPA Scale","Mid-Semester Grade","Final Grade","Default Grade","Added  
By","Added On","Confirmed","Waitlist Position","Waitlist Rank","Waitlisted  
By","Waitlisted On","Dropped By","Dropped On","Roster As Of Date"  
"F16","15688","B","Y","N","Kolter","Zico","","zkolter","zkolter@andrew.cmu.edu","S  
CS","CS","50","12.0","L","4+"," ","","","reg","1 Jun  
2016","Y","","","","","","","","","30 Aug 2016 4:34"
```

- If values themselves contain commas, you can enclose them in quotes (our registrar apparently always does this, just to be safe)

```
import pandas as pd  
dataframe = pd.read_csv("CourseRoster_F16_15688_B_08.30.2016.csv",  
                        delimiter=',', quotechar='"')
```

JSON files / string

- JSON originated as a way of encapsulating Javascript objects
- A number of different data types can be represented
 - Number: 1.0 (always assumed to be floating point)
 - String: "string"
 - Boolean: true or false
 - List (Array): [item1, item2, item3,...]
 - Dictionary (Object in Javascript): {"key":value}
- Lists and Dictionaries can be embedded within each other:
- [{"key": [value1, [value2, value3]]}]

Example JSON data

➤ JSON from Github API

```
{
  'login': 'pvirtue',
  'id': 5945661,
  'node_id': 'MDQ6VXNlcjU5NDU2NjE=',
  'avatar_url': 'https://avatars.githubusercontent.com/u/5945661?v=4',
  'gravatar_id': '',
  'url': 'https://api.github.com/users/pvirtue',
  'html_url': 'https://github.com/pvirtue',
  'followers_url': 'https://api.github.com/users/pvirtue/followers',
  'following_url': 'https://api.github.com/users/pvirtue/following{/other_user}',
  'gists_url': 'https://api.github.com/users/pvirtue/gists{/gist_id}',
  'starred_url': 'https://api.github.com/users/pvirtue/starred{/owner}/{/repo}',
  'subscriptions_url': 'https://api.github.com/users/pvirtue/subscriptions',
  'organizations_url': 'https://api.github.com/users/pvirtue/orgs',
  'repos_url': 'https://api.github.com/users/pvirtue/repos',
  'events_url': 'https://api.github.com/users/pvirtue/events{/privacy}',
  'received_events_url': 'https://api.github.com/users/pvirtue/received_events',
  'type': 'User',
  'site_admin': False,
  'name': 'Pat Virtue', ...
}
```

Parsing JSON in Python

- Built-in library to read/write Python objects from/to JSON files

```
import json

# load json from a REST API call
headers = {'Authorization': 'token '+token}
response = requests.get("https://api.github.com/user", headers=headers)
data = json.loads(response.content)

json.load(file) # load json from file
json.dumps(obj) # return json string
json.dump(obj, file) # write json to file
```

Thank You!